

UNIVERSIDAD AUTONOMA GABRIEL RENE MORENO



PRUEBA Prueba Practicing Data Science

Estudiante: Veronica Tancara Casilla

Docente: Msc. Marcelo Choque

Mayo 2026

Ejercicio 1

Una institución de investigación sociológica requiere analizar la relación entre el poder adquisitivo de los pasajeros y su tasa de supervivencia. Para ello, se solicita desarrollar un cliente analítico en Java puro que procese los bloques físicos del archivo del Titanic en el clúster HDFS, extrayendo el promedio de la tarifa abonada, segmentada por la clase del pasajero, únicamente para aquellos individuos que sobrevivieron al naufragio.

1. Ingesta y Distribución de Bloques (Terminal Anfitriona)

Instalar el CLI de kaggle:

```
$ pip3 install kaggle
```

Descargar el data set de titanic requerido:

```
$ kaggle datasets download -d heptapod/titanic --unzip -  
p ./datos_titanic/
```

Descargar y levantar los servicios configurados en el docker-compose.yml

- namenode
- datanode1
- datanode2
- datanode3
- cliente_analitico

```
$ docker compose up -d
```

Copiar el dataset de train_and_test.csv descargado previamente. Y copiarlo al namenode:

```
$ docker cp train_and_test2.csv namenode:/
```

Ingresar al contenedor del name node:

```
$ docker exec -it namenode bash
```

Dentro del namenode, crear el directorio en HDFS

```
/# hdfs dfs -mkdir -p /user/root/titanic/
```

Y subir el dataset train_and_test2.csv al sistema de archivo HDFS:

```
/# hdfs dfs -put train_and_test2.csv /user/root/titanic
```

Verificar el archivo dentro de HDFS y la distribución de bloques:

```
/# hdfs fsck /user/root/titanic/train_and_test2.csv -files -  
blocks -locations
```

```
+ hadoop docker exec -it namenode bash
root@11d4ec087e4b:/# hdfs dfs -mkdir -p /user/root/titanic/
root@11d4ec087e4b:/# hdfs dfs -put train_and_test2.csv /user/root/titanic/
2026-05-29 02:37:25,036 INFO sasl.SaslDataTransferClient: SASL encryption trust check: localhostTrusted = false, remoteHostTrusted = false
root@11d4ec087e4b:/# hdfs fsck /user/root/titanic/train_and_test2.csv -files -blocks -locations
Connecting to namenode via http://namenode:9870/fsck?ugi=root&files=1&blocks=1&locations=1&path=%2Fuser%2Froot%2Ftitanic%2Ftrain_and_test2.csv
FSCK started by root (auth:SIMPLE) from /172.18.0.2 for path /user/root/titanic/train_and_test2.csv at Fri May 29 02:40:55 UTC 2026
/user/root/titanic/train_and_test2.csv 83879 bytes, replicated: replication=3, 1 block(s): OK
0. BP-25483305-172.18.0.2-1780021489139:blk_1073741825_1001 len=83879 Live_repl=3 [DatanodeInfoWithStorage[172.18.0.3:9866,DS-89824ee5-becb-46c2-bb0-a2c0-4abe0f6bf69d,DISK], DatanodeInfoWithStorage[172.18.0.4:9866,DS-9923e565-1ae9-4dc2-8d2d-acec701dd661,DISK]]

Status: HEALTHY
Number of data-nodes: 3
Number of racks: 1
Total dirs: 0
Total symlinks: 0

Replicated Blocks:
Total size: 83879 B
Total files: 1
Total blocks (validated): 1 (avg. block size 83879 B)
Minimally replicated blocks: 1 (100.0 %)
Over-replicated blocks: 0 (0.0 %)
Under-replicated blocks: 0 (0.0 %)
Mis-replicated blocks: 0 (0.0 %)
Default replication factor: 3
Average block replication: 3.0
Missing blocks: 0
Corrupt blocks: 0
Missing replicas: 0 (0.0 %)

Erasure Coded Block Groups:
Total size: 0 B
Total files: 0
Total block groups (validated): 0
Minimally erasure-coded block groups: 0
Over-erasure-coded block groups: 0
Under-erasure-coded block groups: 0
Unsatisfactory placement block groups: 0
Average block group size: 0.0
Missing block groups: 0
Corrupt block groups: 0
Missing internal blocks: 0
FSCK ended at Fri May 29 02:40:55 UTC 2026 in 64 milliseconds

The filesystem under path '/user/root/titanic/train_and_test2.csv' is HEALTHY
root@11d4ec087e4b:/#
```

2. Desarrollo del Algoritmo Analítico (Java)

```
IntelliJ IDEA

AnalisisTitanic.java x

1  import org.apache.hadoop.conf.Configuration;
2  import org.apache.hadoop.fs.FileSystem;
3  import org.apache.hadoop.fs.Path;
4  import org.apache.hadoop.fs.FSDataOutputStream;
5  import java.io.BufferedReader;
6  import java.io.BufferedWriter;
7  import java.io.InputStreamReader;
8  import java.io.OutputStreamWriter;
9  import java.net.URI;
10 import java.util.HashMap;
11 import java.util.Map;
12
13 public class AnalisisTitanic {
14     public static void main(String[] args) {
15         try {
16
17             Configuration conf = new Configuration();
18             URI uriNameNode = new URI("hdfs://namenode:9000");
19             FileSystem fs = FileSystem.get(uriNameNode, conf);
20
21             Path rutaEntrada = new Path("/user/root/titanic/train_and_test2.csv");
22             Path rutaSalida = new Path("/user/root/datos/resultado_tarifas.csv");
23
24             BufferedReader lector = new BufferedReader(new InputStreamReader(fs.open(rutaEntrada)));
25
26             Map<String, double[]> matrizPorClase = new HashMap<>();
27
28             String linea;
29             lector.readLine();
30             while ((linea = lector.readLine()) != null) {
31                 String[] campos = linea.split(",");
32
33                 if (campos.length < 28) continue;
34
35                 if (!campos[27].trim().equals("1")) continue; //indice 27 survived
36
37                 try {
38                     String pclass = campos[21].trim(); // indice 21
39                     double fare = Double.parseDouble(campos[2].trim()); // indice 2
40
41                     matrizPorClase.putIfAbsent(pclass, new double[]{0.0, 0.0});
42                     matrizPorClase.get(pclass)[0] += fare; // sumatoria
43                     matrizPorClase.get(pclass)[1] += 1; // conteo
44
45                 } catch (NumberFormatException e) {
46                     // Registro malformado
47                 }
48             }
49             lector.close();
50
51             FSDataOutputStream flujoSalida = fs.create(rutaSalida, true);
52             BufferedWriter escritor = new BufferedWriter(
53                 new OutputStreamWriter(flujoSalida)
54             );
55
56             escritor.write("pclass,promedio_tarifa,total_sobrevivientes\n");
57
58             for (Map.Entry<String, double[]> entrada : matrizPorClase.entrySet()) {
59                 String clase = entrada.getKey();
60                 double suma = entrada.getValue()[0];
61                 double conteo = entrada.getValue()[1];
62                 double promedio = (conteo > 0) ? (suma / conteo) : 0.0;
63
64                 escritor.write(
65                     clase + "," +
66                     String.format("%.2f", promedio) + "," +
67                     (int) conteo + "\n"
68                 );
69             }
70
71             escritor.close();
72             fs.close();
73
74             System.out.println("Analisis completado. Resultado en: " + rutaSalida);
75
76         } catch (Exception e) {
77             System.err.println("Error critico: " + e.getMessage());
78             e.printStackTrace();
79         }
80     }
81 }
```

3. Compilación y Despliegue (Terminal del Contenedor)

3.1 Entrar al contenedor del cliente_analitico

```
bash
docker exec -it cliente_analitico bash
```

3.2 Verificar que el script está disponible

```
bash
ls /opt/scripts/
```

3.3 Compilar enlazando el classpath de Hadoop

```
Bash
javac -cp $(hadoop classpath) AnalisisTitanic.java
```

3.4 Empaquetar en JAR

```
Bash
hadoop jar AnalisisTitanic.jar AnalisisTitanic
```

```
Learn more at https://docs.docker.com/go/abag-etc/
+ hadoop docker cp AnalisisTitanic.java cliente_analitico:/opt/scripts
Successfully copied 4.61kB to cliente_analitico:/opt/scripts
+ hadoop docker exec -it cliente_analitico bash
root@9152ad8561fc:/# cd opt/scripts/
root@9152ad8561fc:/opt/scripts# javac -cp $(hadoop classpath) AnalisisTitanic.java
root@9152ad8561fc:/opt/scripts# jar cf AnalisisTitanic.jar AnalisisTitanic.class
root@9152ad8561fc:/opt/scripts# hadoop jar AnalisisTitanic.jar AnalisisTitanic
2026-05-29 03:13:03,707 INFO sasl.SaslDataTransferClient: SASL encryption trust check: localhostTrusted = false, remoteHostTrusted = false
2026-05-29 03:13:05,624 INFO sasl.SaslDataTransferClient: SASL encryption trust check: localhostTrusted = false, remoteHostTrusted = false
Analisis completado. Resultado en: /user/root/datos/resultado_tarifas.csv
```

1. Evaluación de la reducción dimensional

Leer el archivo de resultado desde HDFS

```
bash
# Desde dentro del contenedor cliente_analitico
hadoop fs -cat /user/root/titanic/resultado_tarifas.csv
```

Salida esperada (3 matrices de salida — una por clase socioeconómica):

```
pclass,promedio_tarifa,total_sobrevivientes
1,95.65,136
2,22.05,87
3,13.67,119
```

```
root@9152ad8561fc:/opt/scripts# hadoop fs -cat /user/root/datos/resultado_tarifas.csv
2026-05-29 03:14:22,466 INFO sasl.SaslDataTransferClient: SASL encryption trust check: localhostTrusted = false, remoteHostTrusted = false
pclass,promedio_tarifa,total_sobrevivientes
1,95.61,136
2,22.06,87
3,13.69,119
root@9152ad8561fc:/opt/scripts#
```

Ejercicio 2

El Ministerio de Turismo de una metrópolis ha comisionado la creación de un pipeline de datos (Data Pipeline) para monitorear el mercado de alojamientos a corto plazo. Se le solicita diseñar un flujo que extraiga datos crudos, aplique reglas de calidad empresarial y genere un modelo dimensional agregando funciones matemáticas avanzadas.

Arquitectura de Almacenamiento (Capa Bronze)

1. Descargar el dataset de Airbnb NYC

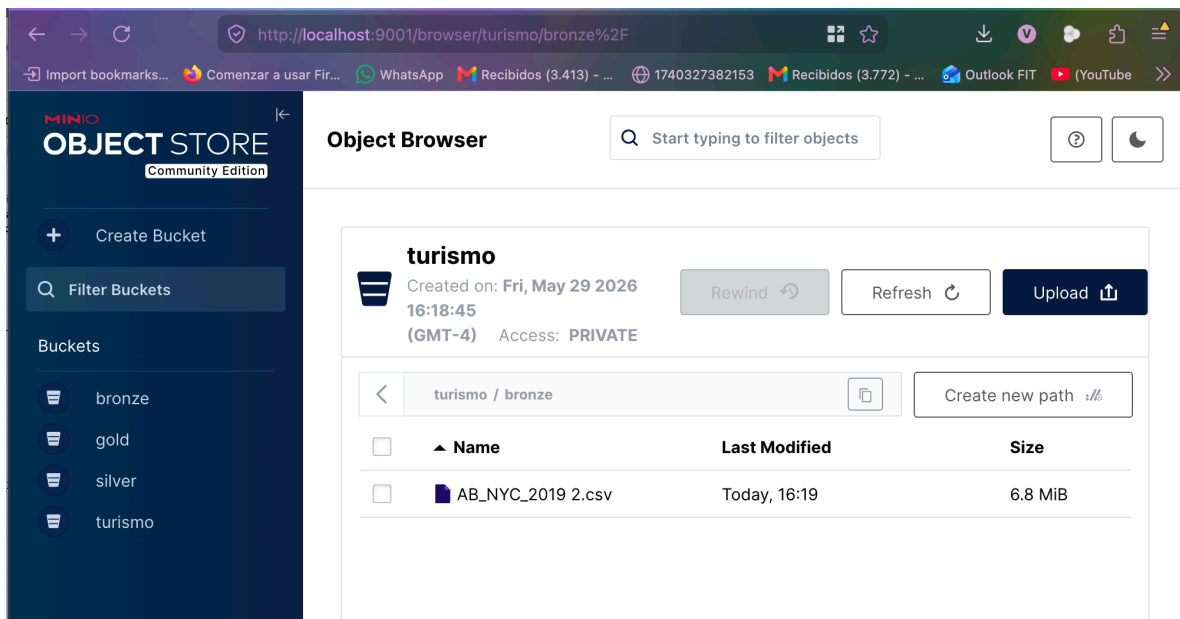
```
kaggle datasets download -d dgomonov/new-york-city-airbnb-open-data \
-p ./data --unzip
```

2. Ejecutar el Docker con los servicios de:

- Minio-datalake
- Spark-worker
- Juoyter-lab
- Spark-master

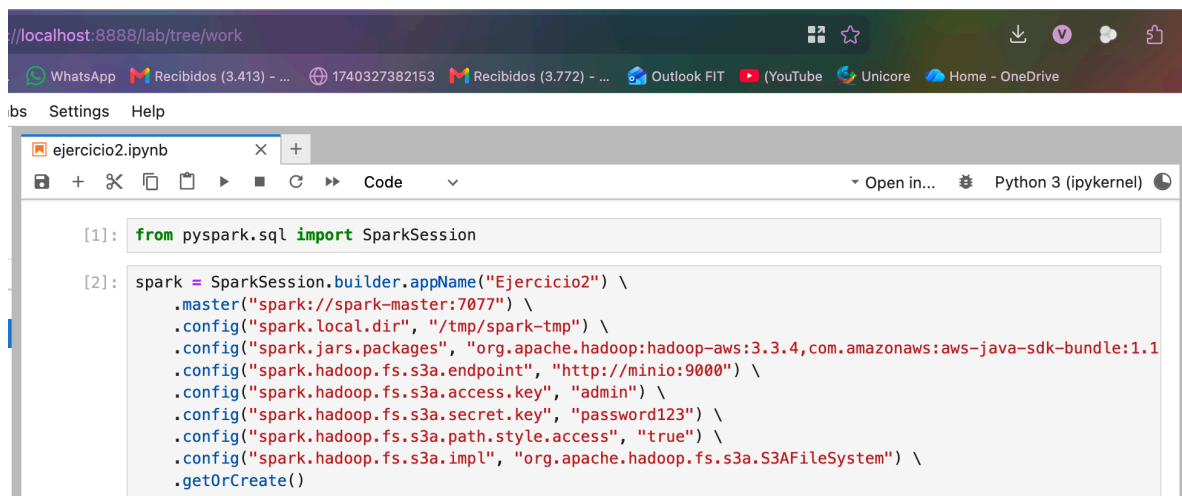
```
$ docker compose up -d
```

Autenticarse en minio y crear los buckets turismo y transferir el archivo AB_NYC_2019.csv directamente a la ruta lógica turismo/bronze/.



Inicialización del Grafo Acíclico Dirigido (Notebook)

1. **Delegación de Computo:** Asegurar que el procesamiento no ocurra en el contenedor de Jupyter, apuntando el contexto lógicamente al nodo spark://spark-master:7077.
2. **Inyección de Dependencias:** Declarar explícitamente los paquetes JAR de hadoop-aws y aws-java-sdk-bundle para habilitar la interoperabilidad de red.
3. **Configuración de Protocolo:** Establecer los parámetros del protocolo s3a:// (Endpoint, Access Key, Secret Key) apuntando al servidor MinIO.



```
[1]: from pyspark.sql import SparkSession

[2]: spark = SparkSession.builder.appName("Ejercicio2") \
    .master("spark://spark-master:7077") \
    .config("spark.local.dir", "/tmp/spark-tmp") \
    .config("spark.jars.packages", "org.apache.hadoop:hadoop-aws:3.3.4,com.amazonaws:aws-java-sdk-bundle:1.11.859") \
    .config("spark.hadoop.fs.s3a.endpoint", "http://minio:9000") \
    .config("spark.hadoop.fs.s3a.access.key", "admin") \
    .config("spark.hadoop.fs.s3a.secret.key", "password123") \
    .config("spark.hadoop.fs.s3a.path.style.access", "true") \
    .config("spark.hadoop.fs.s3a.impl", "org.apache.hadoop.fs.s3a.S3AFileSystem") \
    .getOrCreate()
```

Funciones de Ventana (Spark SQL)

1. **Limpieza** Descarte cualquier registro cuyo precio (price) sea menor o igual a 0, y que posea estrictamente menos de 10 reseñas (number_of_reviews).
2. **Agregación Básica:** Agrupe el conjunto de datos resultante por distrito y tipo de habitación. Calcule el volumen total de alojamientos y el precio promedio por noche.
3. **Álgebra Analítica** Incorpore una función de partición analítica para clasificar jerárquicamente qué tipo de habitación es la más costosa *dentro de cada distrito particular*. El resultado debe ordenarse lógicamente por distrito y luego por su rango de precio.

Persistencia Dimensional (Capa Silver)

Redacte la instrucción final de la API de DataFrames para materializar el resultado de la consulta SQL anterior en el sistema de almacenamiento.1. La escritura debe apuntar a la ruta s3a://turismo/silver/estadisticas_distrito.parquet.

3. Debe forzar la compresión del motor utilizando el formato columnar

```
[*]: df_bronze = spark.read \
    .option("header", "true") \
    .option("inferSchema", "true") \
    .option("quote", "\"") \
    .option("escape", "\\") \
    .option("multiline", "true") \
    .csv("s3a://turismo/bronze/AB_NYC_2019.csv")

df_bronze.createOrReplaceTempView("vista_airbnb")

print(f"Filas en Bronze: {df_bronze.count():,}")
df_bronze.printSchema()
df_bronze.show(5, truncate=False)
```

```
[*]: query_silver = """
WITH datos_limpios AS (
    SELECT neighbourhood_group, room_type, price, number_of_reviews
    FROM vista_airbnb
    WHERE price > 0
        AND number_of_reviews >= 10
),
agregado AS (
    SELECT neighbourhood_group      AS distrito,
           room_type                AS tipo_habitacion,
           COUNT(*)                 AS total_alojamientos,
           ROUND(AVG(price), 2)     AS precio_promedio
    FROM datos_limpios
    GROUP BY neighbourhood_group, room_type
)
SELECT distrito, tipo_habitacion, total_alojamientos, precio_promedio,
       RANK() OVER (
           PARTITION BY distrito
           ORDER BY precio_promedio DESC
       ) AS rango_precio
FROM agregado
ORDER BY distrito, rango_precio
"""

df_silver = spark.sql(query_silver)
df_silver.show(50, truncate=False)
```

```
query_silver = """
WITH datos_limpios AS (
    SELECT neighbourhood_group, room_type, price, number_of_reviews
    FROM vista_airbnb
    WHERE price > 0
        AND number_of_reviews >= 10
),
agregado AS (
    SELECT neighbourhood_group      AS distrito,
           room_type                AS tipo_habitacion,
           COUNT(*)                 AS total_alojamientos,
           ROUND(AVG(price), 2)     AS precio_promedio
    FROM datos_limpios
    GROUP BY neighbourhood_group, room_type
)
SELECT distrito, tipo_habitacion, total_alojamientos, precio_promedio,
       RANK() OVER (
           PARTITION BY distrito
           ORDER BY precio_promedio DESC
       ) AS rango_precio
FROM agregado
ORDER BY distrito, rango_precio
"""

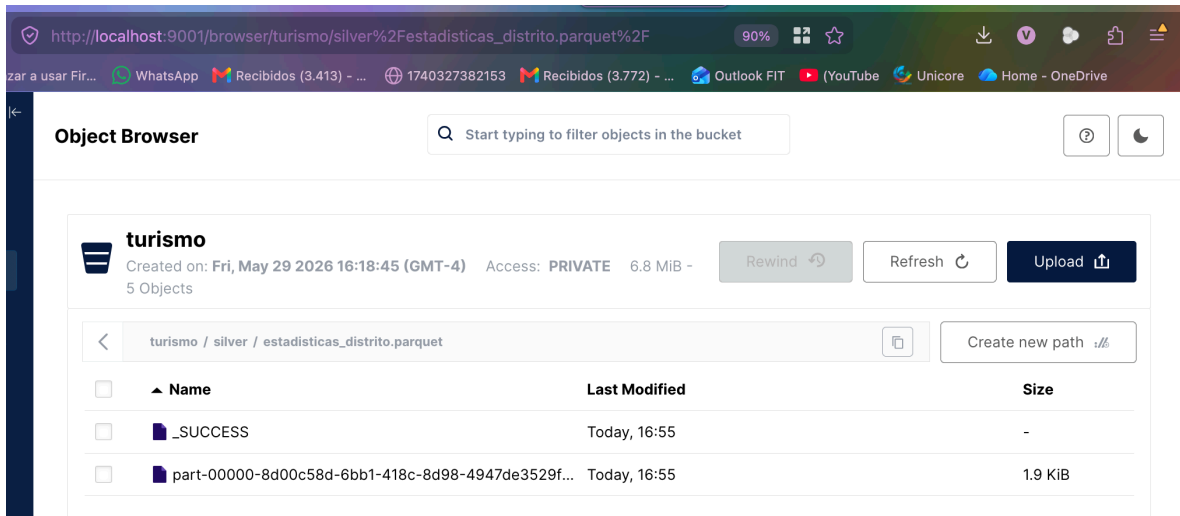
df_silver = spark.sql(query_silver)
df_silver.show(50, truncate=False)
```

Parquet.


```
df_silver.write \
    .mode("overwrite") \
    .parquet("s3a://turismo/silver/estadisticas_distrito.parquet")

print("Capa Silver materializada en s3a://turismo/silver/estadisticas_distrito.parquet")
```

Capa Silver materializada en s3a://turismo/silver/estadisticas_distrito.parquet



Resultado Dimensional (Capa GOLD)

Obtener un reporte final en csv en capa gold con algún reporte correspondiente que usted vea conveniente

```
[6]: df_silver.createOrReplaceTempView("silver_airbnb")
```

```
query_gold = """
WITH extremos AS (
    SELECT distrito,
           MAX(CASE WHEN rango_precio = 1 THEN tipo_habitacion END) AS habitacion_mas_cara,
           MAX(CASE WHEN rango_precio = 1 THEN precio_promedio END) AS precio_max,
           MIN(precio_promedio) AS precio_min,
           SUM(total_alojamientos) AS total_alojamientos_distrito
    FROM silver_airbnb
    GROUP BY distrito
)
SELECT distrito,
       habitacion_mas_cara,
       precio_max,
       precio_min,
       ROUND(precio_max - precio_min, 2) AS diferencia_precio,
       ROUND((precio_max - precio_min) / precio_min * 100, 2) AS dispersion_pct,
       total_alojamientos_distrito
FROM extremos
ORDER BY dispersion_pct DESC
"""
```

```
df_gold = spark.sql(query_gold)
df_gold.show(truncate=False)
```

```
# Persistir como CSV (un solo archivo gracias a coalesce(1))
df_gold.coalesce(1) \
    .write \
    .mode("overwrite") \
    .option("header", "true") \
    .csv("s3a://turismo/gold/reporte_dispersión_precios")
```

```
print("Capa Gold generada en s3a://turismo/gold/reporte_dispersión_precios")
```

distrito	habitacion_mas_cara	precio_max	precio_min	diferencia_precio	dispersion_pct	total_alojamientos_distrito
Brooklyn	Entire home/apt	164.19	42.29	121.9	288.25	8217
Manhattan	Entire home/apt	229.42	74.08	155.34	209.69	7807
Queens	Entire home/apt	134.45	45.52	88.93	195.36	2614
Bronx	Entire home/apt	118.17	43.08	75.09	174.3	525
Staten Island	Entire home/apt	116.25	53.45	62.8	117.49	206

Capa Gold generada en s3a://turismo/gold/reporte_dispersión_precios

```
[6]: df_silver.createOrReplaceTempView("silver_airbnb")

query_gold = """
WITH extremos AS (
    SELECT distrito,
           MAX(CASE WHEN rango_precio = 1 THEN tipo_habitacion END) AS habitacion_mas_cara,
           MAX(CASE WHEN rango_precio = 1 THEN precio_promedio END) AS precio_max,
           MIN(precio_promedio) AS precio_min,
           SUM(total_alojamientos) AS total_alojamientos_distrito
    FROM silver_airbnb
    GROUP BY distrito
)
SELECT distrito,
       habitacion_mas_cara,
       precio_max,
       precio_min,
       ROUND(precio_max - precio_min, 2) AS diferencia_precio,
       ROUND((precio_max - precio_min) / precio_min * 100, 2) AS dispersion_pct,
       total_alojamientos_distrito
FROM extremos
ORDER BY dispersion_pct DESC
"""

df_gold = spark.sql(query_gold)
df_gold.show(truncate=False)

# Persistir como CSV (un solo archivo gracias a coalesce(1))
df_gold.coalesce(1) \
    .write \
    .mode("overwrite") \
    .option("header", "true") \
    .csv("s3a://turismo/gold/reporte_dispersion_precios")

print("Capa Gold generada en s3a://turismo/gold/reporte_dispersion_precios")
```

distrito	habitacion_mas_cara	precio_max	precio_min	diferencia_precio	dispersion_pct	total_alojamientos_distrito
Brooklyn	Entire home/apt	164.19	42.29	121.9	288.25	8217

Object Browser

Start typing to filter objects in the bucket

turismo
Created on: Fri, May 29 2026 16:18:45 (GMT-4) Access: PRIVATE 6.8 MiB - 5 Objects

Rewind Refresh Upload

turismo / gold / reporte_dispersion_precios

Name	Last Modified	Size
_SUCCESS	Today, 16:55	-
part-00000-f26cad6d-7401-420d-abd1-6f5773fa111e-c000.csv	Today, 16:55	390.0 B

distrito	habitacion_mas_cara	precio_max	precio_min	diferencia_precio	dispersion_pct	total_alojamientos_distrito
Brooklyn	Entire home/apt	164.19	42.29	121.9	288.25	8217
Manhattan	Entire home/apt	229.42	74.08	155.34	209.69	7807
Queens	Entire home/apt	134.45	45.52	88.93	195.36	2614
Bronx	Entire home/apt	118.17	43.08	75.09	174.3	525
Staten Island	Entire home/apt	116.25	53.45	62.8	117.49	206

The screenshot shows the MinIO Object Browser web interface. At the top, there's a navigation bar with tabs for 'MinIO Console', 'JupyterLab', and others. Below the navigation bar, the browser address bar shows 'http://localhost:9001/browser/turismo'. The main content area is titled 'Object Browser' and features a search bar with the placeholder text 'Start typing to filter objects in the bucket'. Below the search bar, there's a section for the bucket 'turismo', which includes a trash icon, creation time 'Fri, May 29 2026 16:18:45 (GMT-4)', access level 'PRIVATE', size '6.8 MiB', and a count '1 Object'. To the right of this section are buttons for 'Rewind', 'Refresh', and 'Upload'. Below this, there's a table listing the objects in the bucket. The table has columns for 'Name', 'Last Modified', and 'Size'. The only object listed is a folder named 'bronze'.